

Classifying the Findings

Quality concern	Reported findings	What it means from a software-quality perspective
Naming / readability	<code>invalid-name</code> , <code>redefined-builtin</code> , <code>redefined-outer-name</code>	Names such as <code>T</code> , <code>df</code> , and <code>e</code> do not always clearly communicate their purpose or follow the project's naming conventions. This makes code harder to understand and maintain. The report also shows <code>format</code> redefining a Python built-in.
Documentation	<code>missing-module-docstring</code> , <code>missing-class-docstring</code>	Some modules and classes do not explain their purpose. This increases the effort required for a new developer to understand and maintain the system. For example, missing module docstrings appear in several repository and model files.
Complexity	<code>too-many-locals</code> , <code>no-else-return</code>	Some functions contain more local variables or unnecessary control-flow structures than desirable. This can make logic harder to follow, understand, and test. <code>export_utils.py</code> , for example, has a function with 16 local variables.
Function design	<code>too-many-arguments</code> , <code>unused-argument</code>	Functions receiving many parameters are often harder to call, understand, test, and maintain. The report includes functions with 6, 9, and even more arguments. It also reports an unused <code>student_id</code> argument.
Class design	<code>too-many-instance-attributes</code> , <code>no-self-use</code>	A class with many attributes may be handling too many responsibilities. For example, the <code>Assignment</code> class has 14 instance attributes where Pylint's threshold is 7. Methods that do not use <code>self</code> may also indicate that some behavior does not really belong inside the class.

Maintainability	<code>protected-access</code> , <code>no-else-return</code> , <code>unnecessary-pass</code>	Repeated access to protected members such as <code>_id</code> , <code>_subject</code> , <code>_schedule</code> , etc. creates stronger dependencies between classes. Changes to one class can therefore require changes in other classes.
Error handling	<code>bare-except</code> , <code>broad-except</code> , <code>raise-missing-from</code>	Catching every possible exception can hide the real cause of failures. The report contains both bare exception handling and <code>except Exception</code> . Missing exception chaining can also make debugging harder because the original cause is less clearly preserved.
Unused / dead code	<code>unused-variable</code> , <code>unused-import</code> , <code>unused-argument</code>	Variables or imports that are never used add unnecessary code and make the program harder to understand. For example, <code>workbook</code> is unused, and several imports such as <code>Optional</code> , <code>Union</code> , <code>Type</code> , <code>Path</code> , and <code>Workbook</code> are reported as unused.
Possible defect	<code>syntax-error</code> , <code>import-error</code> , <code>no-name-in-module</code> , <code>unsubscriptable-object</code>	These are more serious because they may indicate code that cannot run correctly. For example, Pylint reports a syntax error in <code>models.admin</code> , an inability to import <code>pandas/openpyxl</code> , and an <code>unsubscriptable-object</code> issue in <code>assignment.py</code> .
Coding convention / formatting	<code>line-too-long</code> , <code>trailing-whitespace</code>	These issues usually do not change the program's behavior, but they reduce consistency and readability. The report contains a very large number of trailing-whitespace findings and several lines exceeding the 100-character limit.
Encapsulation	<code>protected-access</code>	Repository classes frequently directly access members beginning with <code>_</code> , such as <code>_id</code> , <code>_email</code> , <code>_role</code> , and <code>_schedule</code> . This weakens encapsulation because classes are depending directly on another class's internal implementation.

**Dependency /
environment
problems**

`import-error`

Pylint cannot resolve dependencies such as `pandas` and `openpyxl`. This may indicate missing packages or an incorrectly configured development environment. It should therefore be distinguished from a purely stylistic code-quality problem.